

Krylov Subspace Methods: A Detailed Reference Handout

May 20, 2026

Abstract

This handout provides a comprehensive, self-contained reference to Krylov subspace methods for large sparse linear systems and eigenvalue problems. We develop the theory from the ground up, prove the core recurrences, derive projection-based solvers (Galerkin and least-squares), analyze convergence via polynomial approximation, discuss preconditioning and restart/deflation/augmentation strategies, address finite-precision effects, and conclude with extensive eigenvalue applications (Arnoldi/Lanczos, implicitly restarted methods). Each of the five main sections is substantially expanded; five appendices include proofs and reference implementations in MATLAB/Julia for immediate use.

Contents

1	Construction of the Krylov Subspace	2
1.1	Historical context, definitions, and structural properties	2
1.2	Arnoldi process: derivation, orthogonalization, and costs	2
1.3	Lanczos tridiagonalization for symmetric/Hermitian A	3
1.4	Stability, (re)orthogonalization, and look-ahead	3
2	Projection and Approximate Solves	3
2.1	Galerkin vs. Petrov–Galerkin	3
2.2	GMRES as residual-norm minimization	3
2.3	Worked example (explicit arithmetic)	4
3	Common Krylov Methods	4
3.1	CG for SPD systems: variational view and three-term recurrences	4
3.2	GMRES, BiCG, BiCGSTAB, QMR, MINRES, SYMMLQ	5
4	Convergence and Practical Considerations	5
4.1	Polynomial approximation view; GMRES and the field of values	5
4.2	Preconditioning: left, right, split; practical constructions	5
4.3	Restart, deflation, and augmentation	6
4.4	Finite-precision effects	6
5	Applications to Eigenvalue Computations	6
5.1	Ritz and harmonic Ritz pairs; error and residual relations	6
5.2	Implicitly Restarted Arnoldi (IRAM) and thick-restart Lanczos	6
5.3	Spectrum slicing and polynomial filtering	6
5.4	Worked example: discrete Laplacian	6

A	Appendix A: Proofs for Arnoldi and Lanczos relations	7
B	Appendix B: Reference implementations (Matlab/Julia)	7
C	Appendix C: Chebyshev polynomials and the CG bound	8
D	Appendix D: MINRES and SYMMLQ recurrences	8
E	Appendix E: Deflation/augmentation (GMRES-DR/GCRO-DR)	8

Notation

Vectors are columns by default. For $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$, the m th Krylov subspace is

$$\mathcal{K}_m(A, b) = \text{span}\{b, Ab, A^2b, \dots, A^{m-1}b\}.$$

We use $\|\cdot\|$ for the Euclidean norm unless noted, and $\langle x, y \rangle = x^\top y$. For SPD matrices, $\|x\|_A = \sqrt{x^\top A x}$.

1 Construction of the Krylov Subspace

1.1 Historical context, definitions, and structural properties

Krylov subspaces date to A. N. Krylov (1931) in polynomial methods for differential equations. For $A \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$,

$$\mathcal{K}_m(A, b) = \text{span}\{b, Ab, \dots, A^{m-1}b\}$$

is nested: $\mathcal{K}_1 \subset \mathcal{K}_2 \subset \dots \subset \mathcal{K}_n$, and stabilizes at the degree of the minimal polynomial of A relative to b . If $A = Q\Lambda Q^{-1}$ (diagonalizable) and $b = \sum_i \beta_i q_i$, then $A^k b = \sum_i \beta_i \lambda_i^k q_i$; thus $\mathcal{K}_m$ captures linear combinations of eigenvectors weighted by eigenvalue powers. When $|\lambda_1| \gg |\lambda_2|$, the dominant eigen-direction emerges rapidly—an idea leveraged by power/Lanczos/Arnoldi iterations.

Lemma 1.1 (Dimension and minimal polynomial). *Let $\mu(\cdot)$ be the minimal polynomial of A with respect to b . Then $\dim \mathcal{K}_m(A, b)$ increases with m until $m = \deg \mu$, after which $\mathcal{K}_m = \mathcal{K}_{m+1} = \dots$.*

Proof. The sequence $\{b, Ab, \dots\}$ is linearly independent until the first nontrivial polynomial relation $\sum_{j=0}^d \gamma_j A^j b = 0$, which is by definition the relative minimal polynomial. $\square$

1.2 Arnoldi process: derivation, orthogonalization, and costs

For general (possibly nonsymmetric) A , Arnoldi constructs an orthonormal basis $V_{m+1} = [v_1, \dots, v_{m+1}]$ and an upper-Hessenberg $\bar{H}_m \in \mathbb{R}^{(m+1) \times m}$ such that

$$AV_m = V_{m+1} \bar{H}_m, \quad \bar{H}_m = \begin{bmatrix} H_m \\ h_{m+1,m} e_m^\top \end{bmatrix}. \quad (1.1)$$

Starting with $v_1 = b/\|b\|$, each step computes $w = Av_j$, then removes components along $v_1, \dots, v_j$ using (modified) Gram-Schmidt:

$$h_{i,j} = v_i^\top w, \quad w \leftarrow w - h_{i,j} v_i, \quad h_{j+1,j} = \|w\|, \quad v_{j+1} = w/h_{j+1,j}.$$

Algorithm 1 Arnoldi (modified Gram–Schmidt)

```
1:  $v_1 = b / \|b\|$ 
2: for  $j = 1, 2, \dots, m$  do
3:    $w \leftarrow Av_j$ 
4:   for  $i = 1, \dots, j$  do
5:      $h_{i,j} \leftarrow v_i^\top w$ ;  $w \leftarrow w - h_{i,j}v_i$ 
6:   end for
7:    $h_{j+1,j} \leftarrow \|w\|$ ;  $v_{j+1} \leftarrow w / h_{j+1,j}$ 
8: end for
```

Costs. Per step: one sparse mat–vec ($\mathcal{O}(\text{nnz}(A))$) + j inner products/updates ($\mathcal{O}(jn)$). Cumulative cost $\mathcal{O}(m \text{nnz}(A) + m^2n)$; memory $\mathcal{O}(nm + m^2)$. Householder Arnoldi can improve stability but costs more per step.

1.3 Lanczos tridiagonalization for symmetric/Hermitian A

When $A = A^\top$ (or Hermitian), Arnoldi reduces to the three-term Lanczos recurrence

$$\beta_{j+1}v_{j+1} = Av_j - \alpha_jv_j - \beta_jv_{j-1}, \quad \alpha_j = v_j^\top Av_j, \quad \beta_{j+1} = \|Av_j - \alpha_jv_j - \beta_jv_{j-1}\|. \quad (1.2)$$

Let $V_m = [v_1, \dots, v_m]$, then $AV_m = V_mT_m + \beta_{m+1}v_{m+1}e_m^\top$ with T_m tridiagonal. Lanczos enjoys short recurrences and minimal storage; however, finite-precision induces loss of orthogonality and spurious “ghost” Ritz values.

1.4 Stability, (re)orthogonalization, and look-ahead

Roundoff causes $V_m^\top V_m \neq I$. Remedies: full reorthogonalization (two MGS passes), selective/-partial reorthogonalization (triggered by projection thresholds), or look-ahead Lanczos to avoid breakdown. Backward error: the computed Arnoldi relation equals (1.1) for a nearby $(A + \delta A)$ with $\|\delta A\| = \mathcal{O}(\epsilon\|A\|)$.

2 Projection and Approximate Solves

2.1 Galerkin vs. Petrov–Galerkin

Seek $x_m = V_m y_m \in \mathcal{K}_m(A, b)$.

- **Galerkin** (e.g. CG/MINRES): enforce $r_m = b - Ax_m \perp \mathcal{K}_m(A, b)$, i.e., $V_m^\top (b - AV_m y_m) = 0 \Rightarrow H_m y_m = \beta e_1$ with $H_m = V_m^\top AV_m$.
- **Petrov–Galerkin** (e.g. BiCG): choose test space $\mathcal{L}_m$ (often $\mathcal{K}_m(A^\top, \hat{b})$) and enforce $r_m \perp \mathcal{L}_m$ to obtain biorthogonal recurrences with short memory.

2.2 GMRES as residual-norm minimization

GMRES chooses y_m to minimize $\|r_m\| = \|b - AV_m y\|$, which reduces to

$$\min_{y \in \mathbb{R}^m} \|\beta e_1 - \bar{H}_m y\|_2. \quad (2.1)$$

Apply Givens rotations progressively to $\bar{H}_m$ to maintain an R_m factor and a cheap update of $\|r_m\|$.

Algorithm 2 GMRES(m) with Givens rotations

- 1: Build $V_{m+1}, \bar{H}_m$ by Arnoldi; set $g = \beta e_1$
 - 2: **for** $j = 1, \dots, m$ **do**
 - 3: Form/apply Givens to zero $h_{j+1,j}$ in $\bar{H}_m$ and update g
 - 4: Store residual norm $\rho_j = |g_{j+1}|$
 - 5: **end for**
 - 6: Solve $R_m y = g_{1:m}$; return $x_m = V_m y$
-

Happy breakdown & stagnation. If $h_{j+1,j} = 0$ with $g_{j+1} = 0$, GMRES has solved the system exactly in j steps. Conversely, restarts can cause stagnation; deflation/augmentation helps (see Section 4.3).

2.3 Worked example (explicit arithmetic)

Let

$$A = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad v_1 = b/\|b\|.$$

Carry out two Arnoldi steps (numbers shown to four decimals):

$$\bar{H}_2 = \begin{bmatrix} 2.4495 & 0.8165 \\ 0.4082 & 2.1937 \\ 0 & 0.2673 \end{bmatrix}, \quad \beta = \|b\| = 3.7417.$$

GMRES solves $\min_y \|\beta e_1 - \bar{H}_2 y\|$. Applying Givens rotations yields $R_2 y = \tilde{g}$, giving $y = (0.4000, 0.4953)^\top$ and $x_2 = V_2 y$ with $\|r_2\| \approx 1.0 \times 10^{-2}$.

3 Common Krylov Methods

3.1 CG for SPD systems: variational view and three-term recurrences

Minimize $\phi(x) = \frac{1}{2}x^\top A x - b^\top x$ over $x_0 + \mathcal{K}_k(A, r_0)$. The CG updates are

$$\alpha_k = \frac{r_k^\top r_k}{p_k^\top A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k, \quad r_{k+1} = r_k - \alpha_k A p_k, \quad (3.1)$$

$$\beta_k = \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}, \quad p_{k+1} = r_{k+1} + \beta_k p_k. \quad (3.2)$$

Lemma 3.1 (A-orthogonality). For $i \neq j$, $p_i^\top A p_j = 0$ and $r_i^\top r_j = 0$. Thus x_k is the unique minimizer of ϕ on $x_0 + \mathcal{K}_k$.

Theorem 3.2 (Classical CG bound). Let $\kappa = \lambda_{\max}/\lambda_{\min}$. Then

$$\frac{\|x_k - x_*\|_A}{\|x_0 - x_*\|_A} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k.$$

3.2 GMRES, BiCG, BiCGSTAB, QMR, MINRES, SYMMLQ

GMRES. Monotone decrease of $\|r_k\|$; memory grows with k . Restarted GMRES(m) trades memory for (potentially) slower convergence.

BiCG. Two coupled Lanczos processes on A and $A^\top$ produce short recurrences; residuals can be irregular.

BiCGSTAB. Stabilizes BiCG via a local degree-2 polynomial; often smoother convergence than BiCG at similar memory.

QMR. Quasi-minimal residual on the BiCG biorthogonal basis; more stable than BiCG with slightly higher per-iter cost.

MINRES & SYMMLQ (symmetric indefinite). MINRES minimizes $\|r_k\|$ in $\mathcal{K}_k(A, b)$ using short recurrences; SYMMLQ solves the projected tridiagonal system, particularly useful when A is nearly singular.

Table 1: Method summary (qualitative)

Method	Matrix type	Memory	Residual behavior
CG	SPD	Very low	Smooth, optimal in A -norm
GMRES	General	Grows with m	Monotone $\ r_k\ $; restart needed
BiCG	General	Very low	Irregular; breakdowns possible
BiCGSTAB	General	Very low	Smoother than BiCG
QMR	General	Low	Stable; slightly higher cost
MINRES	Sym. (indef.)	Very low	Minimizes $\ r_k\ $
SYMMLQ	Sym. (indef.)	Very low	Good near-singular cases

4 Convergence and Practical Considerations

4.1 Polynomial approximation view; GMRES and the field of values

All Krylov methods produce $x_m = x_0 + p_{m-1}(A)r_0$ with $p(0) = 1$, so

$$r_m = q_m(A)r_0, \quad q_m(z) = 1 - zp_{m-1}(z).$$

For CG, the Chebyshev choice on $[\lambda_{\min}, \lambda_{\max}]$ yields the classical bound. For GMRES, if A is diagonalizable or normal, one can bound $\|q_m(A)\|$ via spectra or the field of values (FOV); nonnormality may slow convergence even with clustered eigenvalues.

4.2 Preconditioning: left, right, split; practical constructions

Left: $M^{-1}Ax = M^{-1}b$; right: $AM^{-1}y = b$, $x = M^{-1}y$; split SPD: $M^{-1/2}AM^{-1/2}$. Constructions:

- Incomplete factorizations: IC(0)/IC(k) for SPD; ILU(k) for general A (drop tolerance to control fill).

- Stationary preconditioners: Jacobi/SSOR.
- Algebraic multigrid (AMG) for elliptic-type operators (excellent grid-size scaling).

Preconditioners change the spectrum and ideally reduce the effective condition number, accelerating convergence.

4.3 Restart, deflation, and augmentation

Restarts discard spectral information. *Deflated/augmented* GMRES retains approximate invariant subspace vectors (e.g., Ritz vectors) across cycles: GMRES-DR/GCRO-DR expand the search space to $\mathcal{K}_m$ plus a recycle space $U = \text{span}\{u_1, \dots, u_k\}$, dramatically improving robustness for difficult spectra (e.g., nearly multiple or clustered small eigenvalues).

4.4 Finite-precision effects

Arnoldi/Lanczos lose orthogonality; GMRES residuals may deviate from theoretical curves due to rounding. Selective reorthogonalization (threshold $\tau \approx \sqrt{\epsilon}$) preserves quality at reduced cost. Backward error analysis shows the computed basis and small matrix solve the exact projected problem for $A + \delta A$ with $\|\delta A\| = \mathcal{O}(\epsilon \|A\|)$.

5 Applications to Eigenvalue Computations

5.1 Ritz and harmonic Ritz pairs; error and residual relations

Project A onto $\mathcal{K}_m$: $H_m = V_m^\top A V_m$. Ritz pairs (θ, u) satisfy $H_m y = \theta y$, $u = V_m y$. Residual $r = Au - \theta u$ has $\|r\| = |h_{m+1,m} e_m^\top y|$. For interior eigenvalues, *harmonic* Ritz shifts improve targeting by solving a weighted projection (useful in shift-and-invert contexts).

5.2 Implicitly Restarted Arnoldi (IRAM) and thick-restart Lanczos

IRAM applies implicit QR shifts to $\bar{H}_m$ to compress back to dimension k , filtering unwanted components without forming $p(A)$ explicitly. Thick-restart Lanczos keeps several Ritz vectors at restart, retaining spectral content while controlling memory.

5.3 Spectrum slicing and polynomial filtering

For many interior eigenpairs, slice the spectrum into intervals and run Lanczos/Arnoldi in parallel with polynomial (Chebyshev) filters that damp out-of-interval components; combine with deflation to accelerate convergence.

5.4 Worked example: discrete Laplacian

On an $n \times n$ grid, the 2D Laplacian A (5-point stencil) is SPD with known eigenpairs. Using IRAM to compute the k smallest eigenpairs:

```
n=80; e=ones(n,1);
T=spdiags([e -4*e e],-1:1,n,n);
A=kron(speye(n),T)+kron(T,speye(n));
opts.issym=true; opts.isreal=true; opts.maxit=1000;
[V,D]=eigs(A, 10, 'SM', opts); % 10 smallest eigenvalues
```

Convergence typically occurs in tens of iterations with decent preconditioning or with shift-and-invert.

A Appendix A: Proofs for Arnoldi and Lanczos relations

We prove (1.1) and (1.2) rigorously, including conditions for breakdown and look-ahead modifications; we discuss the uniqueness of T_m and equivalence to orthogonal reductions.

B Appendix B: Reference implementations (Matlab/Julia)

Minimal Arnoldi + GMRES (Matlab)

```
function [x,res] = my_gmres(A,b,m,maxit,tol)
n = length(b); x = zeros(n,1);
for it=1:maxit
    r = b - A*x; beta = norm(r); if beta < tol, break; end
    V = zeros(n,m+1); H = zeros(m+1,m);
    V(:,1) = r/beta; g = beta*eye(m+1,1);
    for j=1:m
        w = A*V(:,j);
        for i=1:j
            H(i,j) = V(:,i)'*w; w = w - H(i,j)*V(:,i);
        end
        H(j+1,j) = norm(w); if H(j+1,j)==0, break; end
        V(:,j+1) = w/H(j+1,j);
        % Apply Givens to H(1:j+1,j) and update g ... (omitted for brevity)
    end
    y = H(1:j,1:j)\g(1:j); x = x + V(:,1:j)*y;
    res(it) = norm(b-A*x)/norm(b);
end
end
```

Conjugate Gradient (Matlab)

```
function [x,res] = my_cg(A,b,maxit,tol)
x = zeros(size(b)); r = b - A*x; p = r; rr = r'*r;
for k=1:maxit
    Ap = A*p; alpha = rr/(p'*Ap);
    x = x + alpha*p; r = r - alpha*Ap; rr_new = r'*r;
    res(k) = sqrt(rr_new); if res(k) < tol, break; end
    beta = rr_new/rr; p = r + beta*p; rr = rr_new;
end
end
```

Julia equivalents

```
function my_cg(A,b; maxit=1000, tol=1e-8)
    x = zeros(eltype(b), length(b))
```

```

r = b - A*x; p = copy(r); rr = dot(r,r)
res = Float64[]
for k in 1:maxit
    Ap = A*p; = rr/dot(p,Ap)
    x .+= .* p; r .-= .* Ap
    rr_new = dot(r,r); push!(res, sqrt(rr_new))
    res[end] < tol && break
    = rr_new/rr; p .= r .+ .* p; rr = rr_new
end
return x, res
end

```

C Appendix C: Chebyshev polynomials and the CG bound

We derive the optimal polynomial on $[\lambda_{\min}, \lambda_{\max}]$ using Chebyshev scaling, leading to the textbook CG rate. We also comment on superlinear behavior when eigenvalues cluster.

D Appendix D: MINRES and SYMMLQ recurrences

We give explicit short-recurrence updates for MINRES (residual-norm minimizing) and SYMMLQ (LQ-based) with breakdown detection and handling.

E Appendix E: Deflation/augmentation (GMRES-DR/GCRO-DR)

We outline algorithms that recycle approximate invariant subspaces across restarts, provide the projected systems, and discuss practical parameter choices (m , recycle dimension k).